ZAALIMA DEVELOPMENT

Customer Churn Prediction & LTV Engine

Week 3 • Day 6

API Testing, Logging & Error Handling

Make your API production-ready with proper tests, structured logging, and error handling

■ Day Objectives

- ✥ Write API tests using pytest and httpx (async test client)
- ✥ Add structured logging with Python's logging module
- ✥ Implement request/response logging middleware
- ✥ Handle all edge cases: missing fields, invalid values, model errors
- ✥ Create an API documentation README

■ 1. Why Testing Matters

In production, APIs receive unexpected inputs — missing fields, wrong data types, null values. Without tests, these cause silent failures or crashes. **Automated tests** run every time you make a change, catching regressions before deployment.

■■ 2. API Tests

■ tests/test_api.py — API test suite

```
import pytest
from fastapi.testclient import TestClient
```

```
import sys
sys.path.insert(0, '.')
from src.api.main import app

client = TestClient(app)

VALID_CUSTOMER = {
    'gender': 'Male', 'SeniorCitizen': 0, 'Partner': 'No',
    'Dependents': 'No', 'tenure': 24, 'PhoneService': 'Yes',
    'MultipleLines': 'No', 'InternetService': 'Fiber optic',
    'OnlineSecurity': 'No', 'OnlineBackup': 'No',
    'DeviceProtection': 'No', 'TechSupport': 'No',
    'StreamingTV': 'No', 'StreamingMovies': 'No',
    'Contract': 'Month-to-month', 'PaperlessBilling': 'Yes',
    'PaymentMethod': 'Electronic check',
    'MonthlyCharges': 70.35, 'TotalCharges': 1687.65
}

def test_health_endpoint():
    response = client.get('/health')
    assert response.status_code == 200
    assert response.json()['status'] == 'healthy'

def test_predict_valid_customer():
    response = client.post('/predict', json=VALID_CUSTOMER)
    assert response.status_code == 200
    data = response.json()
    assert 'churn_probability' in data
    assert 0.0 <= data['churn_probability'] <= 1.0
    assert data['churn_prediction'] in ['Will Churn', 'Will Stay']
    assert data['risk_level'] in ['High', 'Medium', 'Low']

def test_predict_missing_field():
    bad = {k:v for k,v in VALID_CUSTOMER.items() if k != 'tenure'}
    response = client.post('/predict', json=bad)
    assert response.status_code == 422  # Unprocessable Entity

def test_predict_invalid_tenure():
    bad = {**VALID_CUSTOMER, 'tenure': -5}
    response = client.post('/predict', json=bad)
    assert response.status_code == 422

def test_batch_prediction():
    batch = {'customers': [VALID_CUSTOMER, VALID_CUSTOMER]}
    response = client.post('/predict/batch', json=batch)
    assert response.status_code == 200
    data = response.json()
    assert data['total_customers'] == 2
    assert len(data['predictions']) == 2
```

■ **Run tests**

```
pip install pytest httpx --break-system-packages
pytest tests/test_api.py -v
# Expected: 5 tests passed
```

■ 3. Commit & Checklist

- ■ All 5 API tests pass (pytest tests/test_api.py -v)
- ■ Logging added to /predict endpoint
- ■ 422 errors properly returned for invalid input

■ Week3-Day6 Commit

```
git add .
git commit -m "Week3-Day6: API tests — 5 tests passing, error handling, logging middlewa
re"
git push origin main
```

■ Estimated Time: 5 – 6 hours